

API REST per l'analisi di dati socio-demografici

Stefano Caggegi

Università degli studi di Macerata, Corso di Laurea Triennale in Data Analysis per le Scienze Sociali – Metodi di programmazione avanzata, A.A. 2025/2026

1. Introduzione

L'intero progetto è sviluppato utilizzando il linguaggio di programmazione JavaScript eseguito nell'ambiente di runtime Node.js.

Il servizio web è sviluppato seguendo i principi del paradigma architetturale REST (Representational State Transfer), esponendo una API (Application Programming Interface) basata su 5 endpoint per l'analisi di qualsiasi dataset che si trova nella cartella `data/` del progetto, memorizzato in un file `.json` strutturato come un Array di Oggetti.

Il progetto ha già a disposizione, nella cartella `data/`, tre dataset socio-demografici, `db.json`, `dbtest.json` e `dbtest2.json`. Il file `dbtest.json` è stato utilizzato per testare funzionalità e velocità di risposta su un set di dati più grande (50 mila Oggetti), mentre il file `dbtest2.json` è stato utilizzato per testare il funzionamento su un dataset con campi e valori diversi dai precedenti.

La collection di API testate in Postman è implementata per essere eseguita sul file `db.json` caricato sul server.

I dataset che il server è in grado di gestire devono avere una struttura come nell'esempio seguente:

```
[
  { string: string || number, string: string || number, string: string || number, ... },
  { string: string || number, string: string || number, string: string || number, ... },
  ...
]
```

Il progetto fornisce anche un'applicazione client con un'interfaccia per interagire con il server da terminale.

2. Struttura delle cartelle

```
analisiSocioDemo/
|--- client/
|   |--- client.js
|   |--- utils.js
|--- server/
|   |--- data/
|   |   |--- db.json
|   |   |--- dbtest.json
|   |   |--- dbtest2.json
|   |--- modules/
|   |   |--- apireqbody.js
|   |   |--- middlewares.js
|   |   |--- paths.js
|   |   |--- statistics.js
|   |   |--- test.js
|   |--- server.js
|--- db.postman_collection.json
|--- package.json
|--- package-lock.json
|--- relazione.pdf
```

La struttura è organizzata separando client e server. La cartella `server/` contiene tutti gli script per gestire le funzionalità del server. All'interno della cartella `modules/` si trovano i moduli che implementano funzionalità specifiche (middleware, statistiche, routing), migliorando manutenibilità e riuso del codice, mentre la cartella `data/` contiene i dataset che si vuole fare utilizzare all'applicazione. La cartella `client/` contiene invece il programma che permette all'utente di interagire con il server e il modulo di funzionalità specifiche.

3. Funzionalità del server

È un'applicazione Node.js, contenuta nel file `server.js`, che implementa un web server tramite il framework Express e che espone una API RESTful per l'analisi statistica di un dataset socio-demografico. Oltre a ottenere il contenuto completo, l'elenco dei campi (le "chiavi") o l'elenco di tutte le modalità di un campo del dataset, consente di calcolare delle metriche su un campo selezionato o di raggruppare per un campo e calcolare una metrica sui valori di un altro campo. Si è scelto di consentire il calcolo della moda su tutti i valori, sia numerici che categorici, visto che può essere utile e l'implementazione della funzione lo consente. È comunque possibile limitarne il calcolo ai soli valori categorici tramite una piccola modifica al server con un middleware già disponibile.

Tabella 1

Elenco delle metriche calcolabili sui campi

Nome	Descrizione	Valori accettati	Formato valore restituito
sum	Calcola la somma di tutti i valori di un array	number	number
mean	Calcola la media di tutti i valori di un array	number	number
median	Calcola la mediana di tutti i valori di un array	number	number
min	Calcola il valore minimo degli elementi di un array	number	number
max	Calcola il valore massimo degli elementi di un array	number	number
range	Calcola il range dei valori di un array	number	number
mode	Calcola la moda o più mode di tutti i valori di un array	string number	["nome modalità"]
freqabs	Calcola la frequenza assoluta delle modalità di un array	string number	{ "nome modalità": number }
freqrel	Calcola la frequenza relativa delle modalità di un array	string number	{ "nome modalità": number }
freqperc	Calcola la frequenza percentuale delle modalità di un array	string number	{ "nome modalità": number }
allvalue	Restituisce tutti gli elementi di un array	string number	["nome modalità"]

4. API endpoint

Gli API endpoint richiedono e restituiscono dati in formato JSON. In caso di richiesta formulata in maniera scorretta, il server invia al client un messaggio contenente la motivazione dell'errore nel seguente formato JSON:

```
{ "error": "contenuto del messaggio" }
```

4.1. `http://localhost:3000/dataset`

Questo API endpoint risponde a una richiesta HTTP di tipo GET, non necessita dell'invio di un body da parte dell'utente, e restituisce il contenuto completo del dataset in formato JSON.

4.2. `http://localhost:3000/properties`

Questo API endpoint risponde a una richiesta HTTP di tipo GET, non necessita dell'invio di un body da parte dell'utente, e restituisce un array di tutti i campi presenti nel dataset nel seguente formato JSON:

```
[ "nome_campo" ]
```

4.3. `http://localhost:3000/data/statistics/modality`

Questo API endpoint risponde a una richiesta HTTP di tipo POST e richiede un body JSON nel seguente formato:

```
{ "field": "nome_campo" }
```

Restituisce l'elenco di tutti i valori distinti (modalità) assunti nell'intero dataset dal campo specificato, nel seguente formato JSON:

```
{ "nome_campo": [ "nome_modalità" ] }
```

4.4. <http://localhost:3000/data/statistics/descriptive>

Questo API endpoint risponde a una richiesta HTTP di tipo POST e richiede un body JSON nel seguente formato:

```
{ "field": "nome_campo", "metric": "nome_metrica" }
```

Calcola la metrica sui valori del campo indicato dall'utente e restituisce il risultato nel seguente formato JSON:

```
{ "nome_metrica": valore restituito (vedi tabella 1) }
```

4.5. <http://localhost:3000/data/group>

Questo API endpoint risponde a una richiesta HTTP di tipo POST e richiede un body JSON nel seguente formato:

```
{ "groupBy": "nome_campo", "aggregatedField": "nome_campo", "metric": "nome_metrica" }
```

Raggruppa i dati in base ciascuna modalità del campo indicato in "groupBy" e calcola una metrica sui valori del campo indicato in "aggregatedField". Restituisce il risultato nel seguente formato JSON:

```
{ "nome_modalità": valore restituito (vedi tabella 1) }
```

5. Funzionalità del client

L'applicazione client contenuta nel file `client.js` fornisce una semplice interfaccia da usare su terminale che, tramite un menù iterativo, permette di selezionare il tipo di funzionalità da richiedere al server ed eventualmente compilare i campi per il corretto invio della richiesta, e visualizzare la risposta ricevuta dal server. È implementata per effettuare richieste solo agli API endpoint dell'applicazione `server.js`.

6. Avvio

Per eseguire le applicazioni è necessario disporre dell'ambiente di runtime Node.js. Da riga di comando:

- Posizionarsi all'interno della cartella radice del progetto `analisiSocioDemo/`
- Installare tutte le dipendenze Node del progetto tramite il comando: `npm install`
Questa istruzione crea una cartella `node_modules/` all'interno della cartella radice del progetto e permette di utilizzare tutte le librerie e moduli necessari per eseguire gli script;
- Per avviare il server eseguire il comando: `npm start`
- Per avviare il client, in un secondo terminale, eseguire il comando: `npm run startClient`

6.1. Esempio di utilizzo

Il flusso dell'applicazione prevede che l'utente interagisca con il client, il quale costruisce e invia richieste HTTP al server. Il server valida i parametri ricevuti, elabora il dataset e restituisce una risposta in formato JSON che il client visualizza sul terminale.

La prima operazione da effettuare è l'avvio dell'applicazione server tramite il comando `npm start`. Durante l'avvio viene richiesto all'utente di inserire il nome del dataset (comprensivo di estensione) da caricare (il file deve essere presente all'interno della cartella del progetto `data/`). Se la lettura del file va a buon fine, sul terminale comparirà il messaggio: `Server in ascolto all'indirizzo http://localhost:3000`.

Successivamente, in un secondo terminale, è possibile avviare il client tramite il comando `npm run startClient`. Verrà quindi mostrato un menù contenente l'elenco delle operazioni disponibili.

A titolo esemplificativo, si consideri il caso in cui si voglia calcolare la media del campo "eta" presente nel dataset. Selezionando l'opzione "d", il client mostrerà l'elenco delle metriche disponibili e

richiederà l'inserimento del nome del campo e della metrica da calcolare. Inserendo i valori richiesti, il client invierà al server una richiesta HTTP di tipo POST all'endpoint `/statistics/descriptive` con il seguente body: `{ "field": "eta", "metric": "mean" }`

Il server elaborerà la richiesta e restituirà una risposta in formato JSON, che il client visualizzerà sul terminale, ad esempio: `{ mean: 31.6 }`

7. Sviluppo dell'applicazione server

L'applicazione server è stata realizzata utilizzando il web framework Express.js per Node.js, seguendo lo stile architetturale REST, con separazione tra server, che gestisce il dataset, elabora le richieste e invia le risposte, e client, che inoltra le richieste formulate secondo la documentazione della API e visualizza le risposte del server. Si è scelto di creare API endpoint che gestiscono richieste HTTP di tipo GET per ottenere dati senza elaborazioni da parte dell'applicazione, e richieste HTTP di tipo POST per tutti quei dati che, per l'elaborazione, richiedono l'invio di informazioni da parte dell'utente, in modo da standardizzarne l'invio e l'utilizzo da parte delle funzioni del server tramite il body del Request Object.

Per rendere la manutenibilità e lettura del server più agevole si è scelto di separare e raggruppare tutte le funzioni utilizzate in moduli separati. Nel modulo `statistics.js` sono contenute tutte le funzioni che manipolano i dati e generano il risultato, oltre ad alcune funzioni di supporto utilizzate internamente dalle funzioni che invece costituiscono l'API del modulo e che vengono esportate.

Nel modulo `middlewares.js` sono definite tutte le callback utilizzate dall'applicazione come application-level middleware. Queste funzioni effettuano la validazione dei dati inseriti dall'utente nel Request Object. In caso di dati non conformi, il middleware invia una risposta al client in formato JSON contenente la descrizione dell'errore, senza passare al routing-level middleware successivo, interrompendo la catena dei middleware senza invocare `next()`.

Il modulo `paths.js` definisce ed esporta un oggetto immutabile che raccoglie tutti gli endpoint utilizzati nell'applicazione, così da centralizzarne la gestione e facilitarne la modifica. Mentre il modulo `apiRequestBody.js` definisce ed esporta un oggetto immutabile con tutti i formati dei request body degli API endpoints.

Per lo sviluppo delle funzioni del progetto si è inizialmente adottato un approccio di tipo imperativo, per poi evolvere verso uno stile più funzionale, facendo uso di funzioni di ordine superiore, tecniche di currying e function factory, al fine di generalizzarne il comportamento e ottenerne la riusabilità. La fase di studio, testing e miglioramento delle funzioni è stata effettuata tramite lo script `test.js`.

In generale, quasi tutte le funzioni del modulo `statistics.js` accettano come argomento un array di valori, standardizzando così l'input delle funzioni che calcolano metriche sui dati. Questa scelta deriva anche dalla necessità di implementare un sistema di caching interno al server, così da evitare di accedere e iterare sull'intero dataset a ogni richiesta agli endpoint. In particolare, la funzione `cacheDbByFields(db)` crea un oggetto in cui le chiavi rappresentano i campi degli oggetti del dataset e i rispettivi valori sono array contenenti tutti i valori associati a ciascun campo. Restituisce a sua volta una funzione che prende come argomento il nome di un campo e restituisce l'array dei valori corrispondenti. In questo modo, nello script `server.js`, le callback degli endpoint possono ottenere rapidamente i dati su cui applicare le metriche invocando la variabile `serverCache` (che contiene l'espressione `cacheDbByFields(db)`) con il nome del campo desiderato. Se la cache contiene già i valori relativi a quel campo, questi vengono restituiti immediatamente, altrimenti il dataset viene iterato, i valori vengono estratti, salvati in cache e restituiti.

Il server è implementato come un'unica funzione asincrona `server()` con tutte le istruzioni inserite all'interno di un blocco `try`, avente come prima istruzione la lettura e il caricamento in memoria del dataset selezionato e come ultima istruzione l'avvio del web server. In questa maniera, se la lettura

del dataset non dovesse andare a buon fine il server non verrebbe avviato e, tramite il blocco `catch`, verrebbe visualizzato il messaggio di errore sul terminale. È stato scelto quindi che sia l'utente che avvia il server a selezionare il dataset su cui l'applicazione dovrà operare e fornire le statistiche.

Tutti i middleware gestiscono, inoltre, la generazione di messaggi di log stampati sul terminale in cui è avviato il server. Tali messaggi includono il timestamp della richiesta e una descrizione dell'operazione effettuata, dell'eventuale mancata validazione della richiesta o dell'errore generato.

Per una descrizione completa del comportamento di tutte le funzioni, oggetti e variabili implementati negli script dell'applicazione server è possibile consultare i commenti del codice sorgente.

8. Sviluppo dell'applicazione client

Anche per l'applicazione client si è scelto di raggruppare nel file `utils.js` tutte le funzionalità utilizzate, e implementare nel file `client.js` solo la logica iterativa e la chiamata delle rispettive funzioni, importate tramite modulo, in base alle scelte dell'utente.

Nel modulo `utils.js` sono implementate tutte le funzioni asincrone utilizzate dall'applicazione per effettuare richieste HTTP e visualizzare le risposte. Due funzioni sono impiegate solo internamente al modulo per eseguire le richieste tramite l'API Fetch. La funzione `getFetch(api)` è utilizzata per le richieste di tipo GET, mentre `postFetch(api, reqBody)` è utilizzata per le richieste di tipo POST. Entrambe inviano la richiesta al server, attendono la risposta e restituiscono il body convertito in un oggetto JavaScript. Inoltre, impostano l'header `"Connection": "close"` nel Request Object. Questa scelta è dovuta al comportamento dell'applicazione client, che a ogni iterazione tenta di riutilizzare una connessione già chiusa dal server dopo l'invio della risposta, causando errori di tipo `ECONNRESET`. Impostando tale header si garantisce la chiusura della connessione al termine di ogni richiesta HTTP, evitando così il problema. (Utilizzando il servizio Postman per effettuare le richieste il problema non si verificava). Il modulo esporta cinque funzioni asincrone, una per ciascun endpoint del server. Queste funzioni si occupano di costruire eventualmente il body della richiesta sulla base degli input dell'utente, invocare `getFetch` o `postFetch` con i parametri corretti e visualizzare sul terminale la risposta ricevuta dal server.

L'applicazione client fornisce infine un menù iterativo che permette all'utente di visualizzare l'elenco delle richieste che è possibile inoltrare al server, selezionare l'opzione desiderata, compilare eventuali campi del Request Object e visualizzare la risposta del server.

Per una descrizione completa del comportamento di tutte le funzioni, oggetti e variabili implementati negli script dell'applicazione client è possibile consultare i commenti del codice sorgente.

9. Conclusioni

Il progetto realizzato consente l'analisi flessibile di dataset JSON attraverso un'architettura client-server basata su API REST. La modularizzazione del codice, l'utilizzo di middleware per la validazione delle richieste e l'implementazione di un sistema di caching contribuiscono a migliorare manutenibilità, efficienza e scalabilità del servizio. L'applicazione risulta facilmente estendibile, permettendo l'aggiunta di nuove metriche o funzionalità con modifiche limitate al codice esistente.